

Universidad de Ingeniería y Tecnología – UTEC

CS2023 – Algoritmos y Estructuras de Datos

Periodo 2026-2

Proyecto Final 1

Segment Tree: consultas de suma por rangos
y actualizaciones puntuales

Estructura asignada: **Segment Tree**

Grupo 14

Integrantes

Ormachea Mamani Gabriel

Palomino Meza Ricardo Jesus

Sanchez Echevarria Andre Gianfranco

Profesor: Wilder Nina Choquehuayta

B. Introducción

El objetivo de este proyecto es explicar y demostrar visualmente un Segment Tree implementado desde cero en C++, mediante un video educativo cuya animación proviene de la ejecución real del código. Como caso práctico se usan las ventas por día de un negocio: cada posición del arreglo es un día y se necesita (1) consultar el total vendido en un intervalo de días y (2) corregir el importe de un día concreto, de forma repetida. Recorrer el intervalo en cada consulta cuesta $O(n)$; el Segment Tree reduce consultas y correcciones a $O(\log n)$.

Se usa el ejemplo reproducible del enunciado: arreglo $[2, 1, 5, 3, 4, 7]$; $\text{suma}(1, 4) = 13$; asignar 6 al índice 2; arreglo $[2, 1, 6, 3, 4, 7]$; $\text{suma}(1, 4) = 14$; y el caso borde $\text{suma}(2, 2) = 6$. Los resultados los calcula el programa; no están codificados. Los índices empiezan en 0 y los días en 1: el índice i es el día $i+1$, de modo que el intervalo de índices $[1, 4]$ corresponde a los días 2 a 5.

C. Descripción de la estructura y del TDA

TDA. El tipo de dato abstracto es un arreglo de enteros con tres operaciones: $\text{size}()$, $\text{sum}(l, r)$ (suma de $a[l..r]$, rango inclusivo) y $\text{assign}(i, v)$ ($a[i] \leftarrow v$). La interfaz solo fija *qué* hace cada operación y sus errores; no indica cómo se almacenan los datos ni cuánto cuestan. En el código es la clase abstracta `RangeSumArray`. Una implementación ingenua guarda un vector y recorre el intervalo (sum en $O(n)$, assign en $O(1)$).

Implementación con Segment Tree. Es un árbol binario guardado en un `std::vector<long long>`. Cada nodo representa un intervalo $[l, r]$ y guarda la suma de $a[l..r]$. La raíz (posición 1) representa $[0, n-1]$; un nodo en la posición i con intervalo $[l, r]$ y punto medio $m = l + (r-l)/2$ tiene como hijos a la posición $2i$ ($[l, m]$) y a la $2i+1$ ($[m+1, r]$). Las **hojas** son intervalos de un elemento y guardan $a[i]$; cada **nodo interno** guarda la suma de sus dos hijos (invariante que verifican las pruebas). Con $n = 6$ se usan $2n-1 = 11$ nodos (posiciones 1 a 13 del vector; se reservan $4n$ para cubrir cualquier n con esta numeración).

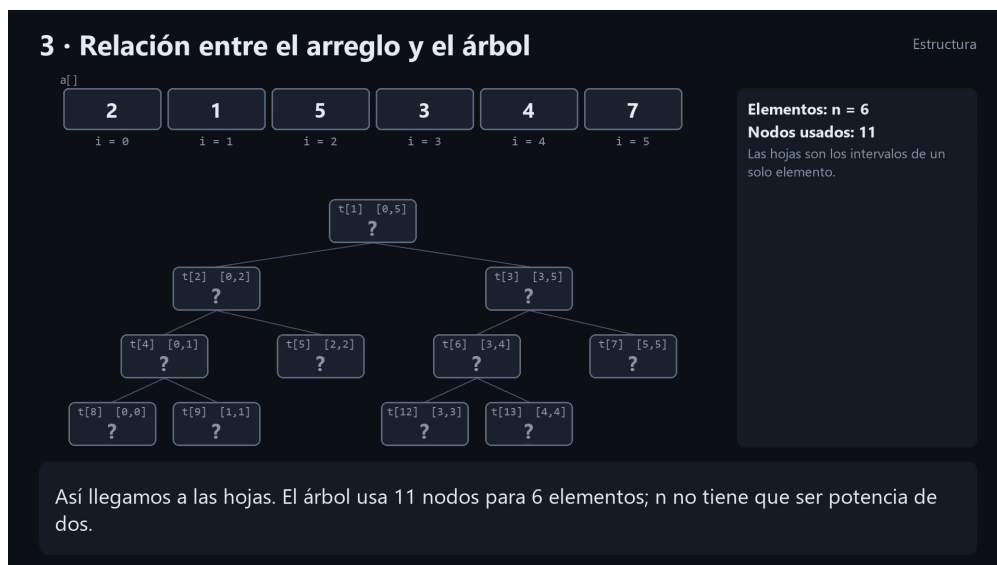


Figura 1. Árbol de intervalos para $n = 6$ antes de construir las sumas (se muestra «?»). Cada nodo indica $t[i]$ y su intervalo $[l, r]$; las hojas quedan sobre su posición del arreglo. (Fotograma extraído del MP4 en $t = 0:57$.)

D. Implementación y demostración

Construcción. Recursión post-orden: se construyen ambos hijos y el padre guarda su suma; cada nodo se calcula una sola vez. **Consulta.** Cada nodo visitado se clasifica frente a $[l, r]$: *nulo* (sin posiciones comunes; aporta 0), *total* (contenido en $[l, r]$; se suma su valor y no se baja) o *parcial* (se visitan ambos hijos). **Actualización.** Se baja por el único camino que contiene el índice, se escribe la hoja y, al volver, se recalcula cada antecesor como suma de sus hijos. **Errores.** Un arreglo vacío es válido ($\text{size}() = 0$); consultarlo o actualizarlo, o usar índices fuera de $[0, n-1]$, lanza `std::out_of_range`; un rango con $l > r$ lanza `std::invalid_argument`.

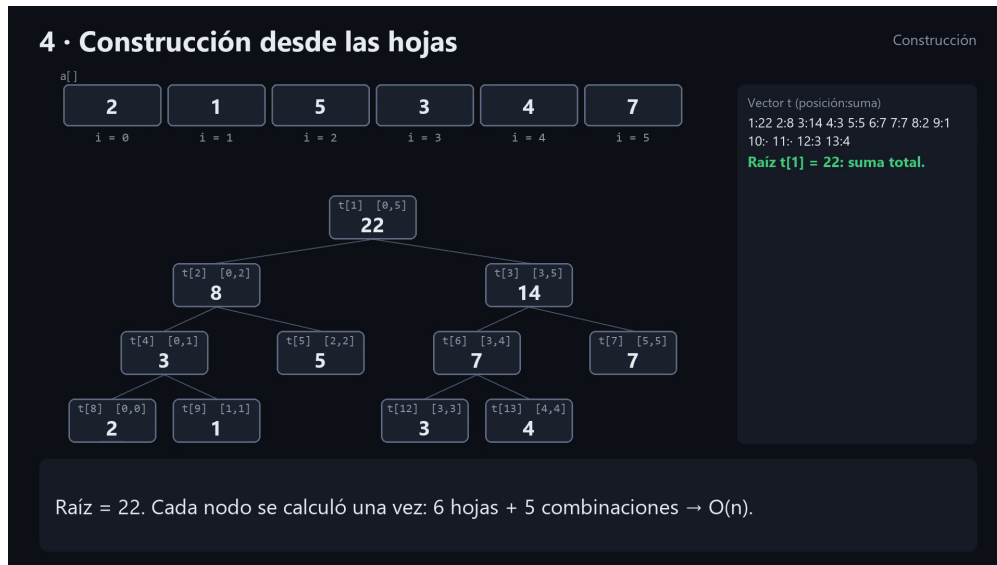


Figura 2. Construcción terminada: la raíz $t[1]$ guarda 22, la suma de todo el arreglo. El panel muestra el vector t (posición:suma). (Fotograma extraído del MP4 en $t = 1:36$.)



Figura 3. Primera consulta suma(1, 4): verde = solapamiento total, ámbar = parcial, gris = nulo. Se combinan $1 + 5 + 7 = 13$ tras visitar 9 nodos. (Fotograma extraído del MP4 en $t = 2:28$.)

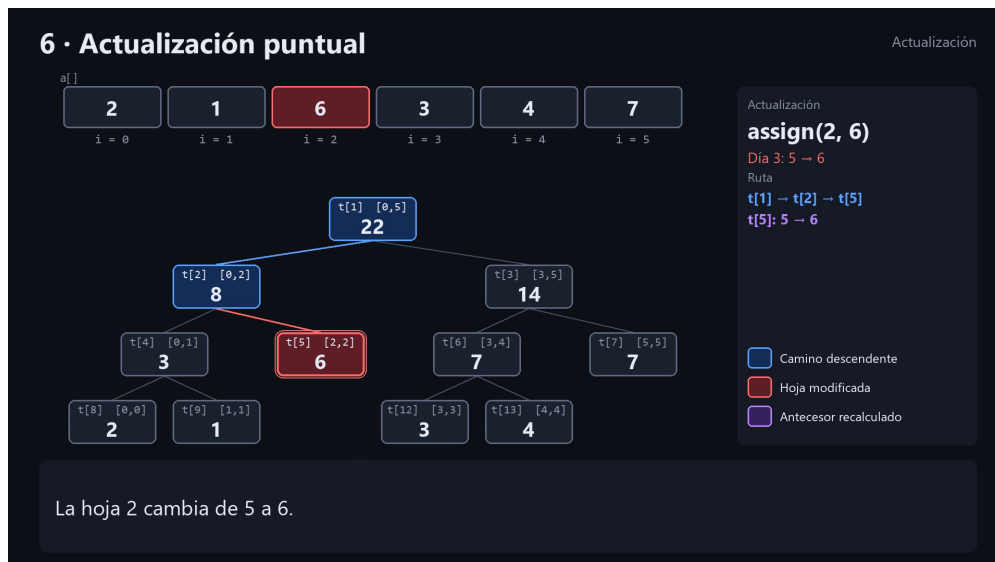


Figura 4. Actualización assign(2, 6): camino descendente en azul, hoja modificada en rojo (5 → 6). Después se recalculan t[2] (8 → 9) y t[1] (22 → 23). (Fotograma extraído del MP4 en t = 2:45.)

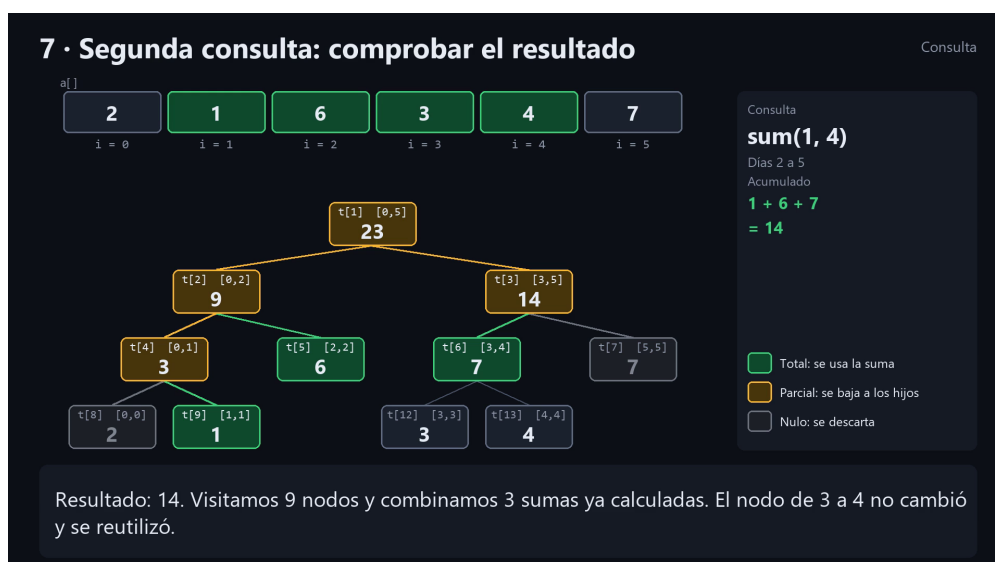


Figura 5. Segunda consulta suma(1, 4) sobre el árbol actualizado: mismo recorrido, resultado 14 (el nodo [3,4] se reutiliza sin cambios). (Fotograma extraído del MP4 en t = 3:24.)

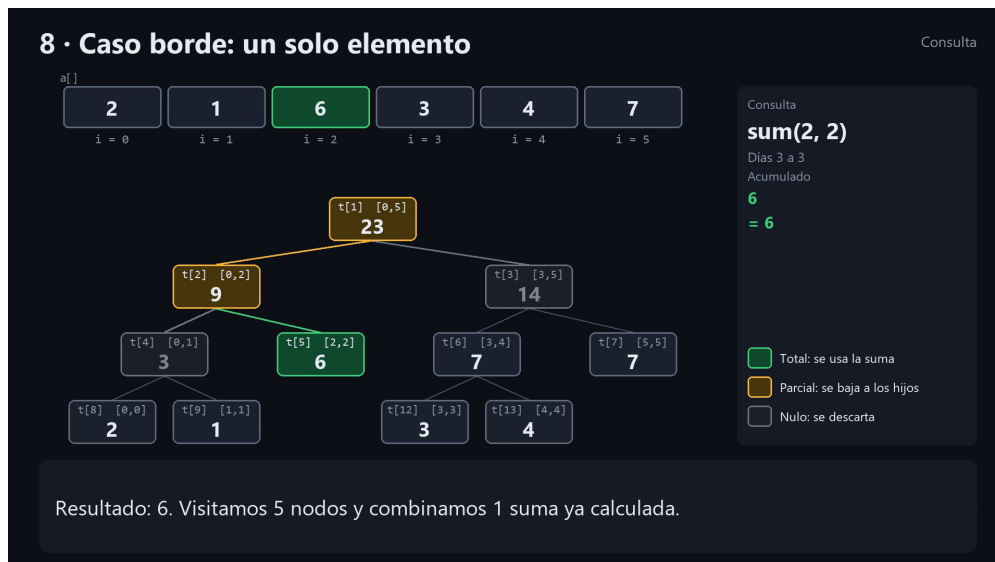


Figura 6. Caso borde suma(2, 2): un intervalo de una sola posición devuelve 6, el valor actualizado, visitando 5 nodos.
(Fotograma extraído del MP4 en $t = 3:56$.)

Caso borde. Además de suma(2, 2), el video muestra la consulta del intervalo completo suma(0, 5) = 23, que visita solo la raíz, y las entradas inválidas (suma(4, 1), suma(0, 6) y una consulta sobre un arreglo vacío) con los mensajes reales de las excepciones.

Cómo los eventos reales de C++ impulsan la animación. SegmentTree recibe un Observer* opcional y lo invoca en cada paso relevante (hoja construida, nodo combinado, nodo visitado con su solapamiento, nodo del camino de actualización, nodo recalculado). La estructura no sabe qué se hace con esas llamadas; TraceRecorder las serializa en trace/trace_demo.json. El animador en Python lee esa traza y solo decide posiciones, colores y transiciones: las sumas, los tipos de solapamiento, las contribuciones y los conteos que se muestran se leen de los eventos y no se recalculan. Antes de animar, trace_model.py comprueba que los eventos sean coherentes (por ejemplo, que reconstruyan el vector t capturado en C++). Las pruebas usan observer = nullptr, así que no generan trazas. El formato de los eventos está documentado en el README.

E. Herramientas y reproducción

Lenguajes y bibliotecas realmente usados: C++17 (solo biblioteca estándar; std::vector como almacenamiento); Python 3.14.5 con Pillow 12.3.0 (dibujo de fotogramas), numpy 2.5.3 (audio), imageio-ffmpeg 0.6.0 (ffmpeg empaquetado, codificación H.264/AAC) y reportlab 5.0.1 (este informe); compilador zig c++ del paquete ziglang 0.16.0 (no había g++ instalado); voz en off con System.Speech de Windows (voz «Microsoft Helena Desktop», es-ES). No se usó Manim.

Comandos exactos, desde la carpeta Proyecto1_Grupo14:

```
pip install -r requirements.txt
python scripts/build.py
build/tests.exe
build/tests_ubsan.exe
build/demo_trace.exe trace/trace_demo.json
python animation/render_video.py --trace trace/trace_demo.json --out Video_Proyecto1_Grupo14.mp4
python scripts/extract_evidence.py
python report/build_report.py
```

scripts/build.py busca g++, clang++ o python -m ziglang c++ y compila con -Wall -Wextra -Wpedantic -Wshadow -Wconversion. El video se renderiza a 1920×1080, 20 fps, y su duración medida es 5:00 (300.2 s).

Pruebas ejecutadas. tests.cpp ejecutó 30 589 comprobaciones, todas superadas, también con UndefinedBehaviorSanitizer: ejemplo completo, consulta de todo el arreglo y de un solo elemento,

arreglo de un elemento, tamaños no potencia de dos, negativos y ceros, actualizaciones repetidas, primer y último elemento, arreglo vacío y entradas inválidas, invariante «nodo = suma de hijos», 300 casos aleatorios con semilla fija comparados con una suma directa (solo en pruebas) y verificación de los eventos exportados. **Limitación:** AddressSanitizer no pudo enlazarse con zig c++ en Windows, por lo que no se ejecutó.

F. Complejidad

Sea n el número de elementos del arreglo y $h = \text{techo}(\log_2 n) + 1$ la altura (número de niveles) del árbol.

Operación	Tiempo	Justificación
Construcción	$O(n)$	Hay $2n-1$ nodos y cada uno se calcula una vez con trabajo constante (copiar una hoja o sumar dos hijos). Construir con n actualizaciones costaría $n \cdot O(\log n) = O(n \log n)$.
$\text{sum}(l, r)$	$O(\log n)$ en el peor caso	Los nodos de un mismo nivel son disjuntos y consecutivos. Solo pueden ser <i>parciales</i> el que contiene la frontera izquierda (entre $l-1$ y l) y el que contiene la derecha (entre r y $r+1$): a lo sumo 2 por nivel. Los nodos totales o nulos no se expanden. Cada parcial visita 2 hijos, luego hay ≤ 4 visitas por nivel y $\leq 4h = O(\log n)$ en total.
$\text{assign}(i, v)$	$O(\log n)$	Recorre un único camino raíz-hoja ($\leq h$ nodos) y hace trabajo constante en cada nodo (bajar, y al volver, recalculando una suma).
$\text{sum}(0, n-1)$ (todo el arreglo)	$O(1)$	La raíz tiene solapamiento total y se devuelve $t[1]$ sin bajar. En general, cualquier consulta que coincide con un nodo desde la raíz es $O(1)$.
Espacio	$O(n)$	$2n-1$ nodos usados; el vector reserva $4n$ posiciones (constante 4). El TDA no necesita almacenar el arreglo aparte.

En la demostración ($n = 6$, $h = 4$) la consulta $\text{suma}(1, 4)$ visitó 9 nodos (cota $4h = 16$), la actualización tocó 3 nodos y la del intervalo completo visitó 1.

Coste de la estructura frente al de la visualización. Las cotas anteriores son de la estructura. Registrar eventos añade trabajo constante por evento, es decir, $O(\log n)$ eventos por consulta o actualización y $O(n)$ durante la construcción; cada captura del vector (snapshot) cuesta $O(n)$ y la demostración toma 2; y renderizar el video cuesta proporcional al número de fotogramas (unos 6 000 a 20 fps), no a n . Por tanto no se atribuye $O(\log n)$ al proceso completo de generar el video.

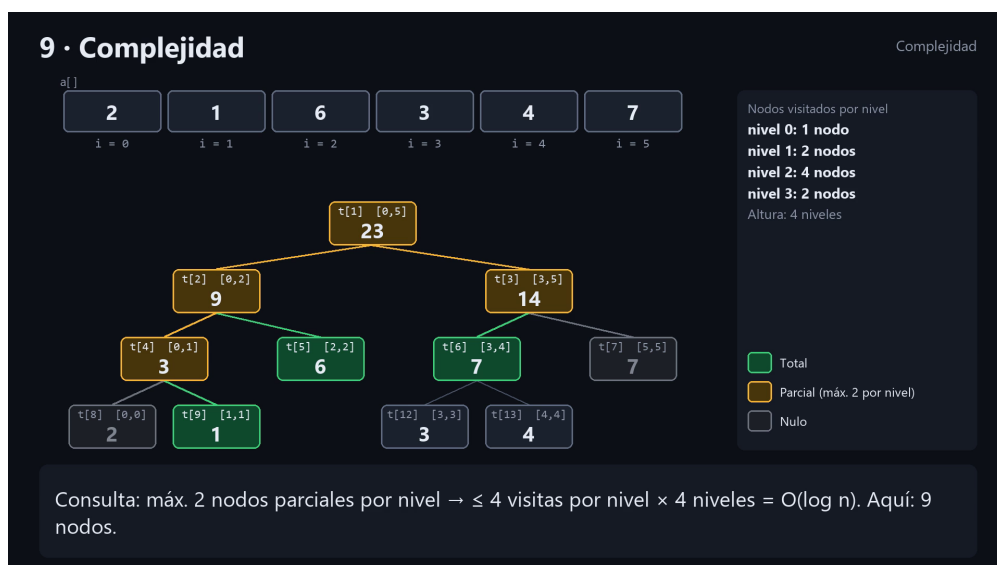


Figura 7. Nodos visitados por nivel en $\text{suma}(1, 4)$ (1, 2, 4 y 2): como máximo dos parciales por nivel. (Fotograma extraído del MP4 en $t = 4:28$.)

G. Evidencias

Las figuras 1 a 7 son fotogramas extraídos del MP4 final con ffmpeg (scripts/extract_evidence.py). Salida real de las pruebas (evidence/salida_pruebas.txt):

```
> build/tests.exe
OK: 30589 comprobaciones superadas
> build/tests_ubsan.exe
OK: 30589 comprobaciones superadas
> build/demo_trace.exe (sin argumentos: sin traza)
sum(1,4) = 13
assign(2, 6)
sum(1,4) = 14
sum(2,2) = 6
sum(0,5) = 23
sum(4,1) -> rango invalido: l > r ([4, 1])
sum(0,6) -> rango fuera de limites: [0, 6] con n = 6
vacio.sum(0,0) -> consulta sobre estructura vacia
```

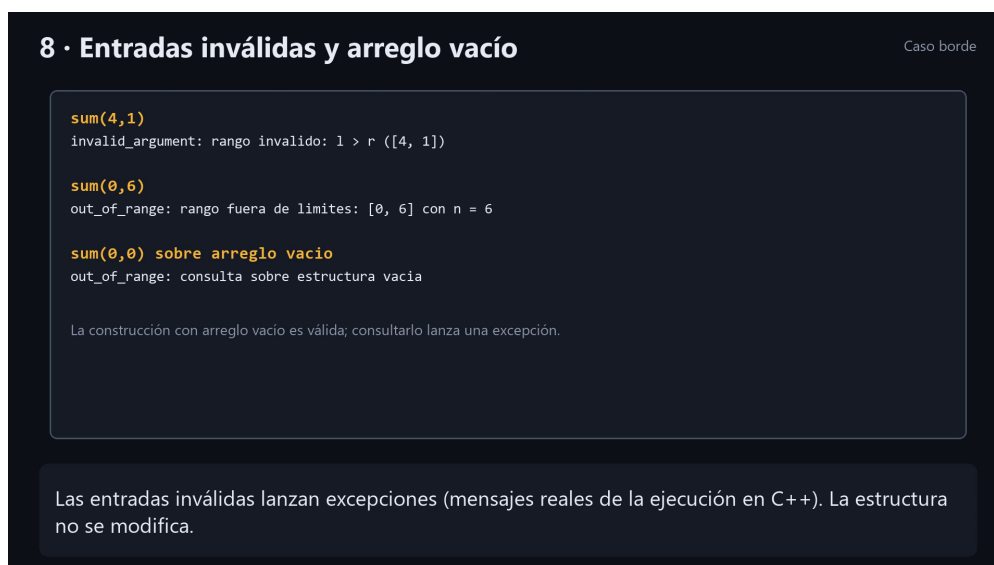


Figura 8. Entradas inválidas y arreglo vacío: los mensajes son los devueltos por las excepciones del código C++ (campo *message* de la traza). (Fotograma extraído del MP4 en $t = 4:17$.)

El video (Video_Proyecto1_Grupo14.mp4, 5:00) se entrega dentro de la carpeta del proyecto. No se incluye un enlace externo porque no se ha publicado ningún archivo.

H. Conclusiones

- Equilibrio entre consultas y actualizaciones.** Frente a la suma directa (consulta $O(n)$, actualización $O(1)$), el Segment Tree ofrece ambas en $O(\log n)$ con espacio $O(n)$. Conviene cuando se mezclan muchas consultas de intervalos con correcciones puntuales, como en el caso de las ventas diarias; si casi no hay consultas, la suma directa es más simple.
- La cota logarítmica de la consulta depende de la clasificación de nodos.** No basta con que el árbol tenga altura $O(\log n)$: la demostración usa que en cada nivel hay como máximo dos nodos parciales, y que los nodos con solapamiento total se resuelven sin bajar. Verlo en el ejemplo (9 nodos visitados, 3 sumas combinadas) hizo concreta esa idea.
- Separar la estructura del registro de eventos permitió una animación fiel.** Con un observador opcional, el mismo código sirve para las pruebas (sin trazas) y para exportar los eventos que mueven la animación; las cifras del video salen de la ejecución y se verifican contra copias del vector.

Limitaciones. Suma en long long sin control de desbordamiento; sin lazy propagation ni actualizaciones de rango (fuera del alcance); voz sintética disponible solo en Windows;

AddressSanitizer no se pudo ejecutar.

I. Referencias

[1] Universidad de Ingeniería y Tecnología (UTEC). *Proyecto Final 1, CS2023 – Algoritmos y Estructuras de Datos, 2026-2*. Enunciado (PDF) del profesor Wilder Nina Choquehuayta.

[2] Video de ejemplo de un tema distinto (Suffix Tree), recibido como archivo local «WhatsApp Video 2026-09-17 at 9.37.58 AM.mp4»; autoría desconocida. Solo se observó su estilo general (fondo oscuro, leyenda de colores, texto explicativo inferior); su contenido y código no se usaron.

[3] Software utilizado: Pillow 12.3.0, numpy 2.5.3, imageio-ffmpeg 0.6.0 (ffmpeg), reportlab 5.0.1 y ziglang 0.16.0 (zig c++), instalados con pip; voz System.Speech de Windows.

El algoritmo del Segment Tree corresponde al conocimiento estándar del curso; durante el desarrollo no se consultó bibliografía adicional ni se copió código de terceros. Si el grupo empleó apuntes del curso o un texto de referencia, debe añadirlo aquí.